Selling Computer Shop

Description

You're the owner of a computer shop facing a financial crisis, and you need to sell off your inventory in a way that yields the highest total profit. An interested buyer has exactly **two** vehicles, each constrained by a fixed load capacity **(Car 1 has capacity car1_load and Car 2 has capacity car2_load)**. You have **N** items available for sale, each with a specific weight (**loads[i]**) and a price (**prices[i]**).

Goal:

Maximize the total selling price of the items loaded into both cars without exceeding each vehicle's capacity.

Requirements:

- Inputs:
  - **car1_load**: Maximum load capacity for Car 1.
  - **car2_load**: Maximum load capacity for Car 2.
  - **N**: Number of items to consider.
  - **loads[]:** Array of integer weights corresponding to each item.
  - **prices[]:** Array of integer prices corresponding to each item.
- Output:
  - The **maximum** total price achievable by optimally packing items into the two cars.

Notes:

- All weights and prices are integers.
- A valid solution must not exceed either car's load capacity.
- You can only include each item **once**.

Complexity

complexity of your algorithm should run in **polynomial time**

Function:

```
static public int RequiredFunction(int car1_load, int car2_load, int
                    N, int[] loads, int[] prices)
```

`PROBLEM_CLASS.cs` includes this method.

Examples

Test Case 1:

N = 4

car1_load = 5

car2_load = 2

loads = { 1, 2, 3, 5 }

prices = { 20, 10, 15, 25 }

Output = 45

Test Case 2:

N = 5

car1_load = 7

car2_load = 10

loads = { 7, 3, 4, 5, 3 }

prices = { 42, 12, 40, 25, 40 }

Output = 134

Test Case 3:

N = 6

car1_load = 6

car2_load = 9

loads = { 2, 5, 9, 6, 5, 4 }

prices = { 10, 12, 30, 15, 20, 25 }

Output = 65

Test Case 4:

N = 5

car1_load = 5

car2_load = 7

loads = { 2, 5, 3, 6, 4 }

prices = { 10, 20, 15, 18, 12 }

Output = 47

Test Case 5:

N = 7

car1_load = 10

car2_load = 8

loads = { 2, 3, 5, 5, 9, 9, 7 }

prices = { 15, 18, 25, 20, 28, 30, 35 }

Output = 93

C# Help

Getting the size of 1D array

```
int size = array1D.GetLength(0);
```

Getting the size of 2D array

```
int size1 = array2D.GetLength(0);

int size2 = array2D.GetLength(1);
```

Creating 1D array

```
int [] array1D = new int [size]
```

Creating 2D array

```
int [,] array2D = new int [size1, size2]
```

Sorting single array

Sort the given array "items" in ascending order

```
Array.Sort(items);
```

Sorting parallel arrays

Sort the first array "master" and re-order the 2nd array "slave" according to this sorting

```
Array.Sort(master, slave);
```